Lecture Note on C++ Programming

By Ikechukwu I. Umeh PhD, FNCS

Topic: Introduction to C++ Programming

---

1. Overview of C++

- **Definition:**
  C++ is a general-purpose programming language that supports procedural, object-oriented, and generic programming paradigms. It was developed by **Bjarne Stroustrup** in 1983 as an extension of the C language, adding object-oriented features.
- **Key Features of C++:**
  - **Object-Oriented Programming (OOP):** Supports classes, objects, inheritance, polymorphism, encapsulation, and abstraction.
  - **High Performance:** Combines low-level memory manipulation with high-level abstractions.
  - **Portability:** C++ programs can run on different platforms with minimal modifications.
  - **Standard Template Library (STL):** Provides reusable components such as algorithms, data structures, and iterators.
  - **Rich Library Support:** Includes libraries for graphics, database connectivity, file handling, and more.

---

2. Structure of a C++ Program

Every C++ program consists of several essential elements. Below is the structure of a basic program:

```
#include <iostream>  // Library for input/output operations

using namespace std;  // Standard namespace

int main() {  // Entry point of the program
    cout << "Hello, World!" << endl;  // Output statement
    return 0;  // Indicates successful execution
}
```

Explanation:

- `#include <iostream>`: Includes the library for input and output operations.
- `using namespace std;`: Simplifies access to standard library functions.
- `int main()`: The main function where the program execution starts.
- `cout <<`: Prints output to the console.

- `return 0;`: Indicates successful termination of the program.

---

3. Data Types and Variables

Data Types in C++:

1. **Basic Types:**
   - `int`: Integer type (e.g., 10, -5).
   - `float`: Floating-point type (e.g., 3.14).
   - `double`: Double-precision floating-point type.
   - `char`: Single character type (e.g., 'A').
   - `bool`: Boolean type (`true` or `false`).
2. **Derived Types:**
   - `array`, `pointer`, `function`.
3. **User-Defined Types:**
   - `class`, `struct`, `enum`.

Variable Declaration:

Variables are containers for storing data. In C++, variables must be declared with a type before use.

```
int age = 25;         // Integer variable
float pi = 3.14;      // Floating-point variable
char grade = 'A';     // Character variable
bool isHappy = true; // Boolean variable
```

---

4. Input and Output in C++

Input using `cin`:

The `cin` object is used to take input from the user.

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;  // Takes input from the user
    cout << "Your age is " << age << endl;
    return 0;
}
```

Output using `cout`:

The `cout` object is used to display output.

```
cout << "Hello, C++!" << endl;  // Prints text to the console
```

5. Control Flow in C++

C++ provides constructs to control the flow of execution:

a. Conditional Statements

if-else Statement:

```
if (condition) {
    // Code if condition is true
} else {
    // Code if condition is false
}
```

Example:

```
int number = 10;
if (number > 0) {
    cout << "Positive number" << endl;
} else {
    cout << "Non-positive number" << endl;
}
```

b. Loops

for Loop:

```
for (int i = 0; i < 5; i++) {
    cout << i << " ";
}
```

while Loop:

```
int i = 0;
while (i < 5) {
    cout << i << " ";
    i++;
}
```

do-while Loop:

```
int i = 0;
do {
    cout << i << " ";
    i++;
} while (i < 5);
```

---

6. Functions in C++

Definition:

A function is a block of code that performs a specific task. Functions help in breaking down the code into reusable modules.

Syntax:

```
return_type function_name(parameters) {
    // Function body
    return value;  // Optional
}
```

Example:

```
#include <iostream>
using namespace std;

int add(int a, int b) {  // Function definition
    return a + b;        // Returns the sum
}

int main() {
    int result = add(5, 3);  // Function call
    cout << "The sum is " << result << endl;
    return 0;
}
```

---

7. Object-Oriented Programming (OOP)

C++ supports object-oriented programming, which is based on the following principles:

a. Classes and Objects

- A **class** is a blueprint for creating objects.
- An **object** is an instance of a class.

Example:

```
class Car {
public:
    string brand;
    int year;

    void display() {
        cout << "Brand: " << brand << ", Year: " << year << endl;
    }
};

int main() {
    Car car1;
    car1.brand = "Toyota";
    car1.year = 2022;
    car1.display();
    return 0;
```

```
}
```

b. Inheritance

Inheritance allows one class to acquire the properties of another.

Example:

```
class Vehicle {
public:
    void start() {
        cout << "Vehicle started" << endl;
    }
};

class Car : public Vehicle {
};

int main() {
    Car myCar;
    myCar.start();  // Inherited method
    return 0;
}
```

c. Polymorphism

Polymorphism allows a single function or operator to perform different tasks.

d. Encapsulation

Encapsulation involves bundling data and methods that operate on the data within a class.

e. Abstraction

Abstraction hides implementation details from the user.

8. Standard Template Library (STL)

Definition:

STL provides pre-built classes and functions for common data structures and algorithms.

Components:

1. **Containers:** Predefined data structures (e.g., vector, list, map).
2. **Algorithms:** Predefined operations (e.g., sort, search).
3. **Iterators:** Objects to traverse containers.

Example:

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int main() {
    vector<int> numbers = {1, 2, 3, 4, 5};
    for (int num : numbers) {
        cout << num << " ";
    }
    return 0;
}
```

---

9. Advantages of C++

- Portable and efficient.
- Supports OOP for better code organization.
- Extensive library support.
- Allows low-level programming for hardware control.

---

10. Applications of C++

- Game development (e.g., Unity, Unreal Engine).
- System software (e.g., operating systems, compilers).
- Embedded systems.
- Financial systems (e.g., trading platforms).

Conclusion

C++ is a powerful, versatile programming language widely used for various applications. By mastering its features, such as object-oriented programming and the Standard Template Library, programmers can build efficient, scalable, and maintainable software systems.

Basic Object-Oriented Concepts in C++

Object-Oriented Programming (OOP) is a paradigm that organizes software design around objects, data, and their behavior. In C++, OOP provides mechanisms like abstraction, objects, classes, methods, parameter passing, and encapsulation to design efficient and maintainable programs.

---

1. Abstraction

> **Definition:** Abstraction is the process of hiding implementation details and exposing only essential features to the user. It focuses on **what an object does** rather than **how it does it**.

Example: When driving a car, we interact with the steering wheel and pedals without worrying about the internal mechanics.

Implementation in C++:

Achieved using **classes** and **abstract classes** (using pure virtual functions).

Example:

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0;  // Pure virtual function
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a Circle" << endl;
    }
};

int main() {
    Shape* shape = new Circle();
    shape->draw();  // Only the interface is exposed
    delete shape;
    return 0;
}
```

---

2. Objects

Definition: An object is an instance of a class. It is a real-world entity with attributes (data) and behaviors (methods).

Example: For a `Car` class, an object `myCar` could have attributes like `color` and `speed` and behaviors like `start()` and `stop()`.

Implementation in C++:

```
class Car {
public:
    string brand;
    int speed;

    void start() {
        cout << "Car is starting..." << endl;
    }
};

int main() {
    Car myCar;
    myCar.brand = "Toyota";
    myCar.speed = 120;
```

C++ Lecture Notes by I.I. Umeh PhD, FNCS: - Not for sale

```
    myCar.start();
    cout << "Brand: " << myCar.brand << ", Speed: " << myCar.speed <<
endl;
    return 0;
}
```

---

3. Classes

Definition: A class is a blueprint for creating objects. It encapsulates data (attributes) and functions (methods) that operate on the data.

Structure of a Class:

```
class ClassName {
private:
    // Private attributes and methods
public:
    // Public attributes and methods
};
```

Example:

```
class Rectangle {
public:
    int length, width;

    int area() {
        return length * width;
    }
};

int main() {
    Rectangle rect;
    rect.length = 10;
    rect.width = 5;
    cout << "Area: " << rect.area() << endl;
    return 0;
}
```

---

4. Methods

Definition: Methods are functions defined within a class that describe the behavior of objects. They operate on the class's data members.

Example with Constructor:

```
class Rectangle {
private:
    int length, width;

public:
    // Constructor to initialize attributes
    Rectangle(int l, int w) {
        length = l;
```

```
        width = w;
    }

    // Method to calculate area
    int area() {
        return length * width;
    }
};

int main() {
    Rectangle rect(10, 5);  // Creating object using constructor
    cout << "Area: " << rect.area() << endl;
    return 0;
}
```

---

5. Parameter Passing

Parameter passing refers to how arguments are passed to functions or methods in C++. There are three main ways:

1. **Pass by Value:**

   A copy of the argument is passed.

   Changes in the method do not affect the original variable.

   Example:

   ```
   void modify(int x) {
       x = 10;
   }

   int main() {
       int a = 5;
       modify(a);
       cout << "a: " << a << endl;  // Output: 5
       return 0;
   }
   ```

2. **Pass by Reference:**

   The actual variable is passed (using references `&`).

   Changes in the method affect the original variable.

   Example:

   ```
   void modify(int& x) {
       x = 10;
   }

   int main() {
       int a = 5;
       modify(a);
   ```

```
    cout << "a: " << a << endl;  // Output: 10
    return 0;
}
```

3. **Pass by Pointer:**

   The address of the variable is passed.

   Example:

```
void modify(int* x) {
    *x = 10;
}

int main() {
    int a = 5;
    modify(&a);
    cout << "a: " << a << endl;  // Output: 10
    return 0;
}
```

---

6. Encapsulation

Definition: Encapsulation is the bundling of data and methods that operate on the data within a class. It restricts direct access to some of the object's components using access specifiers:

Public: Accessible from anywhere.

Private: Accessible only within the class.

Protected: Accessible within the class and derived classes.

Example:

```
class Employee {
private:
    int salary;  // Private data member

public:
    void setSalary(int s) {
        salary = s;  // Setter method
    }

    int getSalary() {
        return salary;  // Getter method
    }
};

int main() {
    Employee emp;
    emp.setSalary(50000);
    cout << "Employee Salary: " << emp.getSalary() << endl;
    return 0;
}
```

Introduction to Strings and String Processing in C++

Strings are sequences of characters used to represent text in C++. C++ provides:

1. **C-Style Strings:** Null-terminated character arrays.
2. **`std::string` Class (Preferred):** Part of the Standard Template Library (STL).

---

1. C-Style Strings

Declared as arrays of characters and terminated with a null character (`\0`).

Example:

```
#include <iostream>
using namespace std;

int main() {
    char name[] = "John";  // Null-terminated character array
    cout << "Name: " << name << endl;
    return 0;
}
```

---

2. `std::string` Class

Provides an easier way to work with strings.

Basic Operations:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Hello";
    string str2 = "World";

    // Concatenation
    string result = str1 + " " + str2;
    cout << "Concatenated String: " << result << endl;

    // Length of String
    cout << "Length: " << result.length() << endl;

    // Accessing Characters
    cout << "First Character: " << result[0] << endl;

    // Substring
    cout << "Substring: " << result.substr(6, 5) << endl;  // "World"

    // Comparison
```

```
        if (str1 == "Hello") {
            cout << "Strings are equal." << endl;
        }

        return 0;
    }
```

String Processing Examples

Problem 1: Reverse a String

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str;
    cout << "Enter a string: ";
    cin >> str;

    string reversed = string(str.rbegin(), str.rend());
    cout << "Reversed String: " << reversed << endl;

    return 0;
}
```

Problem 2: Count Vowels in a String

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str;
    cout << "Enter a string: ";
    cin >> str;

    int vowels = 0;
    for (char c : str) {
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
            c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U') {
            vowels++;
        }
    }

    cout << "Number of vowels: " << vowels << endl;

    return 0;
}
```

Object-oriented concepts like abstraction, objects, classes, methods, parameter passing, and encapsulation provide a structured way to design software. Strings in C++ allow powerful text processing, using both C-style strings and the `std::string` class for ease of use and flexibility. These concepts are fundamental to writing efficient and maintainable C++ programs.

C++ Lecture Notes by I.I. Umeh PhD, FNCS: - Not for sale

Basic C++ Data Types

In C++, data types define the type of data that a variable can store. These are categorized into fundamental, derived, and user-defined data types. Below is a comprehensive explanation of **basic (fundamental) data types** in C++.

---

1. Integer Types

These are used to store whole numbers (both positive and negative).

a. int (Integer):

Definition: Represents standard integer values.
Memory Size: Typically 4 bytes (32 bits) on most platforms.
Range: -2,147,483,648 to 2,147,483,647 (on 32-bit systems).
Example:

```
int age = 25;
```

b. short (Short Integer):

Definition: Used for smaller integer values to save memory.
Memory Size: Typically 2 bytes (16 bits).
Range: -32,768 to 32,767.
Example:

```
short number = 100;
```

c. long (Long Integer):

Definition: Used for larger integer values.
Memory Size: Typically 4 bytes or more.
Range: Depends on the system (e.g., 64-bit systems support larger ranges).

C++ Lecture Notes by I.I. Umeh PhD, FNCS: - Not for sale

Example:
```
long bigNumber = 1000000;
```

d. long long (Long Long Integer):

Definition: Used for very large integer values.
Memory Size: Typically 8 bytes (64 bits).
Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
Example:
```
long long veryBigNumber = 1000000000000;
```

e. unsigned:

Definition: Used to store only positive values (doubles the range by eliminating negative numbers).
Example:
```
unsigned int positiveNumber = 300;
```

---

2. Floating-Point Types

These are used to store numbers with decimal points.

a. float (Floating-Point):

Definition: Used for single-precision floating-point numbers.
Memory Size: Typically 4 bytes.
Range: ±3.4E-38 to ±3.4E+38.
Precision: Up to 7 decimal digits.
Example:
```
float pi = 3.14;
```

b. double (Double Precision Floating-Point):

Definition: Used for double-precision floating-point numbers.
Memory Size: Typically 8 bytes.
Range: ±1.7E-308 to ±1.7E+308.
Precision: Up to 15 decimal digits.
Example:
```
double largeDecimal = 3.14159265359;
```

c. long double (Extended Precision Floating-Point):

Definition: Used for extended precision floating-point numbers.
Memory Size: Typically 10 bytes or more (depends on the system).
Precision: More than `double` (varies by compiler and architecture).
Example:
```
long double preciseValue = 3.141592653589793238 46;
```

---

3. Character Type

a. char (Character):

Definition: Stores a single character or ASCII value.
Memory Size: Typically 1 byte (8 bits).
Range: -128 to 127 (signed) or 0 to 255 (unsigned).
Example:

```
char grade = 'A';
```

Note: Characters are internally stored as integers using their ASCII values.

```
char letter = 'A'; // ASCII value of 'A' is 65.
cout << (int)letter; // Output: 65
```

---

4. Boolean Type

a. bool:

Definition: Stores either `true` or `false`.
Memory Size: Typically 1 byte.
Example:

```
bool isPassed = true;
```

Note: Internally, `true` is represented as `1`, and `false` is represented as `0`.

---

5. Void Type

a. void:

Definition: Represents the absence of a value or no data type. Commonly used for functions that do not return a value.
Example:

```
void displayMessage() {
    cout << "Hello, World!" << endl;
}
```

---

6. Wide Character Type

a. wchar_t (Wide Character):

Definition: Stores a wide character (used for extended character sets like Unicode).
Memory Size: Typically 2 or 4 bytes (depends on the platform).
Example:

```
wchar_t wideChar = L'Ω'; // L prefix indicates a wide character.
```

Summary of Basic Data Types

Data Type	Memory Size	Range	Example
`int`	4 bytes	-2,147,483,648 to 2,147,483,647	`int age = 25;`
`short`	2 bytes	-32,768 to 32,767	`short x = 100;`
`long`	4 bytes (or more)	System-dependent	`long y = 100000;`
`long long`	8 bytes	-9,223,372,036,854,775,808 to ...	`long long z = 1e12;`
`float`	4 bytes	±3.4E-38 to ±3.4E+38	`float pi = 3.14;`
`double`	8 bytes	±1.7E-308 to ±1.7E+308	`double d = 1.2345;`
`long double`	10 bytes or more	More precision than `double`	`long double ld;`
`char`	1 byte	-128 to 127 or 0 to 255	`char c = 'A';`
`bool`	1 byte	`true` (1) or `false` (0)	`bool b = true;`
`wchar_t`	2 or 4 bytes	Platform-dependent	`wchar_t wc = L'Ω';`

Key Notes:

1. **Signed vs. Unsigned:**
    - By default, most numeric types are **signed** (can store both positive and negative values).
    - Adding `unsigned` before a type allows it to store only positive values, effectively doubling the range.

```
2. unsigned int positiveOnly = 100;
```

3. **Type Modifiers:**
    - Modifiers like `short`, `long`, `signed`, and `unsigned` can be combined with basic data types for more flexibility.

```
4. unsigned short smallPositive = 500;
5. long long veryLargeNumber = 9000000000000;
```

C++ basic data types provide the foundation for variables, and understanding their memory size, range, and application is essential for efficient programming.

Writing, Editing, and Running a C++ Program

Writing, editing, and running a C++ program involves several steps. Here is a comprehensive guide to help you through the entire process.

---

1. Writing a C++ Program

Step 1: Setting Up the Development Environment

To write and run a C++ program, you need:

1. A **text editor** or an **Integrated Development Environment (IDE)** such as:
    - Text Editors: Notepad++, Sublime Text, or Visual Studio Code.
    - IDEs: Code::Blocks, Dev-C++, Eclipse, or Microsoft Visual Studio.
2. A **C++ compiler**, such as:
    - GCC (GNU Compiler Collection).
    - MSVC (Microsoft Visual C++).
    - Clang.

Step 2: Writing Your First C++ Program

Here is a simple "Hello, World!" program to get started:

```
#include <iostream> // Includes the input/output library
using namespace std; // Allows the use of standard library functions without
prefixing 'std::'

int main() {
    cout << "Hello, World!" << endl; // Outputs "Hello, World!" to the
console
    return 0; // Indicates that the program has executed successfully
}
```

---

2. Editing a C++ Program

Editing involves modifying the program to meet your requirements or fixing errors. Below are some common tasks:

Adding Functionality

You can extend the "Hello, World!" program by adding user input and more operations:

```
#include <iostream>
using namespace std;

int main() {
    string name;
    cout << "Enter your name: ";  // Asking for user input
    cin >> name;  // Reading input from the user
    cout << "Hello, " << name << "!" << endl;  // Displaying a personalized
message
    return 0;
```

```
}
```

Debugging

If you encounter errors in your program:

1. **Syntax Errors**: Check for missing semicolons, unmatched braces, or typos.
2. **Logic Errors**: Test the program with different inputs to ensure it behaves as expected.
3. **Runtime Errors**: Look for issues like division by zero or accessing invalid memory.

Commenting Your Code

Comments help you and others understand your code better:

```
// This is a single-line comment

/*
This is a
multi-line comment
*/
```

---

3. Running a C++ Program

Step 1: Compile the Program

The C++ source code must be compiled into machine code using a compiler. Below are some ways to do it:

1. **Using GCC (Command Line):**
    - Open your terminal or command prompt.
    - Navigate to the folder where your C++ file is saved.
    - Compile the file:
    - `g++ -o program_name program_name.cpp`
    - This creates an executable file named `program_name`.
2. **Using an IDE:**
    - Open the IDE and create a new project or file.
    - Copy your C++ code into the editor.
    - Use the "Build" or "Run" button in the IDE to compile and run the program automatically.

Step 2: Run the Executable

- If you're using GCC:
    - Run the compiled program by typing:
    - `./program_name`
- If you're using an IDE, it will run the program for you after building.

---

4. Example: Writing, Editing, and Running a Complete C++ Program

Problem Statement:

Write a program to calculate the sum of two numbers entered by the user.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2, sum;

    cout << "Enter the first number: ";
    cin >> num1; // Taking input for the first number

    cout << "Enter the second number: ";
    cin >> num2; // Taking input for the second number

    sum = num1 + num2; // Calculating the sum

    cout << "The sum of " << num1 << " and " << num2 << " is: " << sum <<
endl;

    return 0;
}
```

Steps to Run the Program

1. Save the file as `sum.cpp`.
2. Compile using GCC:
3. `g++ -o sum sum.cpp`
4. Run the executable:
5. `./sum`
6. Enter two numbers as prompted, and the program will display their sum.

---

5. Debugging and Error Handling

Common Errors and Fixes

1. **Syntax Error:**
    - Missing a semicolon at the end of a statement.
    - `cout << "Hello, World!" // Error: Missing semicolon`

      Fix: Add `;` at the end.

2. **Logic Error:**
    - Incorrect implementation of an algorithm.
    - `sum = num1 - num2; // Error: Used subtraction instead of addition`

      Fix: Use `+` for addition.

C++ Lecture Notes by I.I. Umeh PhD, FNCS: - Not for sale

3. **Runtime Error:**
    - Division by zero or invalid input.
    - `int x = 10 / 0; // Error: Division by zero`

    Fix: Validate inputs before performing operations.

6. Tips for Writing Efficient Programs

1. **Modular Code:**
    - Break your program into functions for better readability and reusability.
2. **Error Checking:**
    - Always validate user input to avoid unexpected behavior.
3. **Use Comments:**
    - Document your code to make it easier to understand.

Conclusion

Writng, editing, and running a C++ program is a systematic process that involves:

1. Writing the code with proper syntax and logic.
2. Debugging and testing to fix any errors.
3. Compiling and executing the program to achieve the desired output.

This workflow is essential for developing efficient and error-free C++ applications.

Programming Problems Using Selection in C++

The **selection statements** in C++ (`if`, `if-else`, `if-else if`, and `switch`) are used to make decisions based on conditions. Below are examples of programming problems designed to demonstrate selection statements, along with their implementations in C++.

Problem 1: Determine If a Number is Even or Odd

Problem Statement:

Write a program that reads an integer from the user and determines whether it is even or odd using an `if-else` statement.

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    int number;
    cout << "Enter an integer: ";
    cin >> number;

    if (number % 2 == 0) {
```

```
        cout << number << " is even." << endl;
    } else {
        cout << number << " is odd." << endl;
    }

    return 0;
}
```

Problem 2: Find the Maximum of Two Numbers

Problem Statement:

Write a program that takes two integers as input and determines the larger number using an `if-else` statement.

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2;
    cout << "Enter two integers: ";
    cin >> num1 >> num2;

    if (num1 > num2) {
        cout << num1 << " is greater than " << num2 << "." << endl;
    } else if (num2 > num1) {
        cout << num2 << " is greater than " << num1 << "." << endl;
    } else {
        cout << "Both numbers are equal." << endl;
    }

    return 0;
}
```

Problem 3: Grading System

Problem Statement:

Write a program that takes a student's score as input and assigns a grade based on the following criteria:

- `90-100`: Grade A
- `80-89`: Grade B
- `70-79`: Grade C
- `60-69`: Grade D
- Below 60: Grade F

Implementation:

```
#include <iostream>
```

```
using namespace std;

int main() {
    int score;
    cout << "Enter the student's score (0-100): ";
    cin >> score;

    if (score >= 90 && score <= 100) {
        cout << "Grade: A" << endl;
    } else if (score >= 80 && score < 90) {
        cout << "Grade: B" << endl;
    } else if (score >= 70 && score < 80) {
        cout << "Grade: C" << endl;
    } else if (score >= 60 && score < 70) {
        cout << "Grade: D" << endl;
    } else if (score < 60) {
        cout << "Grade: F" << endl;
    } else {
        cout << "Invalid score entered!" << endl;
    }

    return 0;
}
```

Problem 4: Simple Calculator Using `switch`

Problem Statement:

Write a program that acts as a simple calculator. The program should take two numbers and an operator (+, -, *, /) as input and perform the corresponding operation.

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    double num1, num2;
    char op;

    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    cout << "Enter an operator (+, -, *, /): ";
    cin >> op;

    switch (op) {
        case '+':
            cout << "Result: " << num1 + num2 << endl;
            break;
        case '-':
            cout << "Result: " << num1 - num2 << endl;
            break;
        case '*':
            cout << "Result: " << num1 * num2 << endl;
            break;
        case '/':
```

```
            if (num2 != 0) {
                cout << "Result: " << num1 / num2 << endl;
            } else {
                cout << "Error: Division by zero is not allowed!" << endl;
            }
            break;
        default:
            cout << "Invalid operator!" << endl;
    }

    return 0;
}
```

Problem 5: Determine the Type of a Triangle

Problem Statement:

Write a program that takes the lengths of three sides of a triangle as input and determines its type:

- Equilateral: All three sides are equal.
- Isosceles: Two sides are equal.
- Scalene: All sides are different.

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    int side1, side2, side3;

    cout << "Enter the lengths of three sides of a triangle: ";
    cin >> side1 >> side2 >> side3;

    if (side1 == side2 && side2 == side3) {
        cout << "The triangle is Equilateral." << endl;
    } else if (side1 == side2 || side1 == side3 || side2 == side3) {
        cout << "The triangle is Isosceles." << endl;
    } else {
        cout << "The triangle is Scalene." << endl;
    }

    return 0;
}
```

Problem 6: Leap Year Checker

Problem Statement:

Write a program to check if a given year is a leap year. A year is a leap year if:

- It is divisible by 4.

- However, if it is divisible by 100, it must also be divisible by 400.

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    int year;

    cout << "Enter a year: ";
    cin >> year;

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
        cout << year << " is a leap year." << endl;
    } else {
        cout << year << " is not a leap year." << endl;
    }

    return 0;
}
```

Problem 7: Determine the Eligibility to Vote

Problem Statement:

Write a program that checks if a user is eligible to vote. A person is eligible if their age is 18 or above.

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    int age;

    cout << "Enter your age: ";
    cin >> age;

    if (age >= 18) {
        cout << "You are eligible to vote!" << endl;
    } else {
        cout << "You are not eligible to vote." << endl;
    }

    return 0;
}
```

Problem 8: Check for Positive, Negative, or Zero

Problem Statement:

Write a program that determines if a given number is positive, negative, or zero.

C++ Lecture Notes by I.I. Umeh PhD, FNCS: - Not for sale

Implementation:

```
#include <iostream>
using namespace std;

int main() {
    int number;

    cout << "Enter a number: ";
    cin >> number;

    if (number > 0) {
        cout << "The number is positive." << endl;
    } else if (number < 0) {
        cout << "The number is negative." << endl;
    } else {
        cout << "The number is zero." << endl;
    }

    return 0;
}
```

Conclusion

The above problems illustrate how to use selection statements in C++ to solve practical problems.

Functions, Modules, File I/O, and Basic Error Handling in C++

---

1. Functions in C++

A function is a reusable block of code that performs a specific task. Functions make programs modular and easier to maintain.

Key Components of a Function

1. **Function Declaration (Prototype):**
    - Specifies the function's name, return type, and parameters (optional).
    - Example:
    - `int add(int, int);`
2. **Function Definition:**
    - Contains the actual implementation.
    - Example:
    - `int add(int a, int b) {`
    - `    return a + b;`
    - `}`

3. **Function Call:**
    - Invokes the function.
    - Example:
    - `int result = add(5, 3);`

Example: Simple Function

```
#include <iostream>
using namespace std;

// Function prototype
int add(int, int);

int main() {
    int num1 = 5, num2 = 10;
    cout << "The sum is: " << add(num1, num2) << endl;
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

---

2. Modules in C++

In C++, **modules** can be simulated using **header files** and **source files** to organize code into smaller, reusable components.

Structure of Modules

1. **Header File (.h):**

    Contains declarations of functions, classes, and constants.

    Example: `math_functions.h`

    ```
    // Header file
    int add(int, int);
    int multiply(int, int);
    ```

2. **Implementation File (.cpp):**

    Contains the function definitions.

    Example: `math_functions.cpp`

    ```
    #include "math_functions.h"

    int add(int a, int b) {
        return a + b;
    }
    ```

```
int multiply(int a, int b) {
    return a * b;
}
```

3. **Main Program File:**

   Includes the header file and uses the functions.

   Example: `main.cpp`

```
#include <iostream>
#include "math_functions.h"
using namespace std;

int main() {
    cout << "Addition: " << add(5, 3) << endl;
    cout << "Multiplication: " << multiply(4, 2) << endl;
    return 0;
}
```

---

3. File Input/Output (File I/O)

C++ provides **file streams** (`fstream`, `ifstream`, and `ofstream`) for file operations.

Basic File Operations

1. **Writing to a File:**

   Use `ofstream` to create and write to a file.

   Example:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("example.txt");
    if (outFile.is_open()) {
        outFile << "Hello, C++ File I/O!" << endl;
        outFile << "This is a test file." << endl;
        outFile.close();
        cout << "Data written to file." << endl;
    } else {
        cout << "Unable to open file for writing." << endl;
    }
    return 0;
}
```

2. **Reading from a File:**

   Use `ifstream` to read data from a file.

Example:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream inFile("example.txt");
    string line;
    if (inFile.is_open()) {
        while (getline(inFile, line)) {
            cout << line << endl;
        }
        inFile.close();
    } else {
        cout << "Unable to open file for reading." << endl;
    }
    return 0;
}
```

3. **Appending to a File:**

   Use `ofstream` with the append mode.

   Example:

```
ofstream outFile("example.txt", ios::app);  // Append mode
outFile << "Additional line appended to file." << endl;
```

---

4. Basic Error Handling in C++

Error handling is essential for dealing with unexpected situations during program execution.

a. Using Conditional Checks

Check for conditions like division by zero, invalid input, or file errors.

Example:

```
int divide(int a, int b) {
    if (b == 0) {
        cout << "Error: Division by zero!" << endl;
        return -1;  // Indicating error
    }
    return a / b;
}
```

b. Exception Handling

C++ provides the `try`, `catch`, and `throw` keywords for exception handling.

Syntax:

```
try {
    // Code that might throw an exception
} catch (type exception) {
    // Code to handle the exception
}
```

Example:

```
#include <iostream>
using namespace std;

int divide(int a, int b) {
    if (b == 0) {
        throw runtime_error("Division by zero error");
    }
    return a / b;
}

int main() {
    try {
        int result = divide(10, 0);
        cout << "Result: " << result << endl;
    } catch (runtime_error &e) {
        cout << "Exception caught: " << e.what() << endl;
    }
    return 0;
}
```

c. File Stream Error Handling

Check if a file was successfully opened or processed.

Example:

```
ifstream inFile("nonexistent.txt");
if (!inFile) {
    cerr << "Error: File could not be opened." << endl;
}
```

Examples Combining Concepts

Problem: Save and Retrieve Student Records
Write a program to save a student's name and marks in a file and retrieve the data.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void saveStudentRecord(const string &name, int marks) {
    ofstream outFile("students.txt", ios::app);
    if (outFile.is_open()) {
        outFile << name << " " << marks << endl;
        outFile.close();
        cout << "Record saved!" << endl;
    } else {
```

```
        cerr << "Error: Could not open file for writing!" << endl;
    }
}

void displayStudentRecords() {
    ifstream inFile("students.txt");
    string name;
    int marks;

    if (inFile.is_open()) {
        while (inFile >> name >> marks) {
            cout << "Name: " << name << ", Marks: " << marks << endl;
        }
        inFile.close();
    } else {
        cerr << "Error: Could not open file for reading!" << endl;
    }
}

int main() {
    string name;
    int marks;

    cout << "Enter student's name: ";
    cin >> name;
    cout << "Enter marks: ";
    cin >> marks;

    saveStudentRecord(name, marks);

    cout << "\nStudent Records:" << endl;
    displayStudentRecords();

    return 0;
}
```

Conclusion

- **Functions** improve modularity and reusability of code.
- **Modules** organize code into manageable components using header and source files.
- **File I/O** enables interaction with external files for persistent data storage.
- **Error Handling** ensures robustness by managing unexpected errors during execution.
  These concepts are crucial for building maintainable and efficient C++ applications.

Programming Problems Using Loops

Below are examples of programming problems that demonstrate the use of loops in C++, followed by object-oriented programming (OOP) implementations using classes.

---

1. Programming Problems Using Loops

Problem 1: Print Multiplication Table

Write a program that prints the multiplication table for a number entered by the user.

C++ Lecture Notes by I.I. Umeh PhD, FNCS: - Not for sale

```
#include <iostream>
using namespace std;

int main() {
    int number;
    cout << "Enter a number: ";
    cin >> number;

    for (int i = 1; i <= 10; i++) {
        cout << number << " x " << i << " = " << number * i << endl;
    }

    return 0;
}
```

Problem 2: Sum of Natural Numbers

Write a program to calculate the sum of the first `n` natural numbers using a `while` loop.

```
#include <iostream>
using namespace std;

int main() {
    int n, sum = 0, i = 1;

    cout << "Enter a positive integer: ";
    cin >> n;

    while (i <= n) {
        sum += i;
        i++;
    }

    cout << "The sum of the first " << n << " natural numbers is: " << sum <<
endl;

    return 0;
}
```

Problem 3: Fibonacci Sequence

Write a program to generate the Fibonacci sequence up to a given number of terms.

```
#include <iostream>
using namespace std;

int main() {
    int n, t1 = 0, t2 = 1, nextTerm;

    cout << "Enter the number of terms: ";
    cin >> n;

    cout << "Fibonacci Sequence: ";
    for (int i = 1; i <= n; i++) {
        cout << t1 << " ";
        nextTerm = t1 + t2;
```

```
        t1 = t2;
        t2 = nextTerm;
    }

    cout << endl;
    return 0;
}
```

Problem 4: Check for Prime Numbers

Write a program to check whether a number is prime using a loop.

```
#include <iostream>
using namespace std;

int main() {
    int n, i;
    bool isPrime = true;

    cout << "Enter a positive integer: ";
    cin >> n;

    if (n <= 1) {
        isPrime = false;
    } else {
        for (i = 2; i <= n / 2; i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
    }

    if (isPrime)
        cout << n << " is a prime number." << endl;
    else
        cout << n << " is not a prime number." << endl;

    return 0;
}
```

Problem 5: Pattern Printing

Write a program to print the following pattern for `n` rows:

```
*
**
***
****
*****
#include <iostream>
using namespace std;
```

```
int main() {
    int n;

    cout << "Enter the number of rows: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << endl;
    }

    return 0;
}
```

---

2. Using Classes as Data Abstractions (OOP Approach)

Classes in C++ allow us to encapsulate data and functions, creating a blueprint for objects. Below are problems demonstrating OOP principles with data abstractions.

Problem 1: Class for a Bank Account

Problem Statement:

Design a `BankAccount` class with attributes for account number, account holder name, and balance. Implement methods to deposit, withdraw, and display account details.

Implementation:

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string accountHolder;
    int accountNumber;
    double balance;

public:
    // Constructor
    BankAccount(string name, int number, double initialBalance) {
        accountHolder = name;
        accountNumber = number;
        balance = initialBalance;
    }

    // Deposit method
    void deposit(double amount) {
        balance += amount;
        cout << "Deposited: " << amount << endl;
    }

    // Withdraw method
```

```
    void withdraw(double amount) {
        if (amount > balance) {
            cout << "Insufficient funds!" << endl;
        } else {
            balance -= amount;
            cout << "Withdrawn: " << amount << endl;
        }
    }

    // Display account details
    void displayDetails() {
        cout << "Account Holder: " << accountHolder << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {
    // Create an account
    BankAccount account("John Doe", 12345, 1000.0);

    account.displayDetails();

    account.deposit(500);
    account.withdraw(300);
    account.withdraw(1500);

    account.displayDetails();

    return 0;
}
```

Problem 2: Class for a Student Record

Problem Statement:

Design a `Student` class with attributes for the name, roll number, and marks. Implement methods to input details, calculate the average of marks, and display the record.

Implementation:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int rollNumber;
    double marks[5];

public:
    // Method to input student details
    void inputDetails() {
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter roll number: ";
        cin >> rollNumber;
```

```
        cout << "Enter marks for 5 subjects: ";
        for (int i = 0; i < 5; i++) {
            cin >> marks[i];
        }
    }

    // Method to calculate average marks
    double calculateAverage() {
        double sum = 0;
        for (int i = 0; i < 5; i++) {
            sum += marks[i];
        }
        return sum / 5;
    }

    // Method to display student details
    void displayDetails() {
        cout << "Name: " << name << endl;
        cout << "Roll Number: " << rollNumber << endl;
        cout << "Average Marks: " << calculateAverage() << endl;
    }
};

int main() {
    Student student;

    student.inputDetails();
    student.displayDetails();

    return 0;
}
```

Problem 3: Class for a Rectangle

Problem Statement:

Design a `Rectangle` class with attributes for length and width. Implement methods to calculate the area and perimeter and compare two rectangles for equality.

Implementation:

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length;
    double width;

public:
    // Constructor
    Rectangle(double l, double w) {
        length = l;
        width = w;
    }

    // Method to calculate area
    double calculateArea() {
```

```
        return length * width;
    }

    // Method to calculate perimeter
    double calculatePerimeter() {
        return 2 * (length + width);
    }

    // Method to compare two rectangles
    bool isEqual(Rectangle other) {
        return (length == other.length && width == other.width);
    }
};

int main() {
    Rectangle rect1(10.5, 5.0);
    Rectangle rect2(10.5, 5.0);

    cout << "Area of Rectangle 1: " << rect1.calculateArea() << endl;
    cout << "Perimeter of Rectangle 1: " << rect1.calculatePerimeter() <<
endl;

    if (rect1.isEqual(rect2)) {
        cout << "The two rectangles are equal." << endl;
    } else {
        cout << "The two rectangles are not equal." << endl;
    }

    return 0;
}
```

Conclusion

- **Loops** simplify repetitive tasks, such as generating patterns, performing calculations, or handling user input.
- **Classes and OOP** allow encapsulation and modular design, making code more reusable and maintainable. Through methods and attributes, you can represent real-world entities and abstract complex problems.